

Actividad 4 - Tecnología front end en la construcción de una aplicación web II



ANDREA PILAR TRIANA ROMERO

CORPORACIÓN UNIVERSITARIA IBEROAMERICANA

CALLE 67 # 5 – 27

FACULTAD DE INGENIERÍA

INGENIERÍA DE SOFTWARE

BOGOTÁ D.C

MAYO 2026

Actividad 4 - Tecnología front end en la construcción de una aplicación web II

Andrea Pilar Triana Romero – atrian11@estudiante.ibero.edu.co–100196794



Corporación Universitaria Iberoamericana

Calle 67 # 5 - 27

Facultad de Ingeniería

Ingeniería de Software

Docente: Diana Toquica

IBERO

BOGOTÁ D.C. - CARRANZA 100 - TELÉFONO 312 2000

Bogotá D.C

Mayo 2026

Tabla de contenido

Lista de figuras	3
Introducción	4
REST con Swagger	5
ReactJS.....	7
Hooks.....	9
useState.....	9
useContext.....	10
useEffect.....	11
useReducer.....	12
Context API.....	13
Peticiones HTTP con Axios.....	15
Rutas y navegación	17
Despliegue	19
URL del Código de la Aplicación Web	21
Bibliografía.....	23

IBERO

UNIVERSIDAD DE CANTABRIA



Planeta Formación y Universidades

VIGILADA MINEDUCACIÓN

Lista de figuras

Figura 1. Ejemplo 1.....	6
Figura 2. Ejemplo 2.....	8
Figura 3. Ejemplo 3.....	10
Figura 4. Ejemplo 4.....	11
Figura 5. Ejemplo 5.....	11
Figura 6. Ejemplo 6.....	12
Figura 7. Ejemplo 7.....	14
Figura 8. Ejemplo 8.....	16
Figura 9. Ejemplo 9.....	18
Figura 10. Ejemplo 10.....	20

IBERO

UNIVERSIDAD DE CASTILLA-LA MANCHA



Planeta Formación y Universidades

VIGILADA MINEDUCACIÓN

Introducción

El desarrollo de aplicaciones web requiere integrar diferentes tecnologías que permitan construir interfaces agradables, funcionales y conectadas con servicios internos de procesamiento de información. En este sentido, el front-end cumple un papel fundamental, ya que corresponde a la parte visible con la que interactúa el usuario, mientras que el back-end permite gestionar los datos, las solicitudes y la lógica interna del sistema.

En esta actividad se retoma el proyecto Gamenglish, una propuesta de aplicación web educativa orientada a la enseñanza del inglés en niños, con el fin de analizar algunas tecnologías utilizadas en la construcción de aplicaciones modernas. Para ello, se abordarán conceptos como REST con Swagger, ReactJS, Hooks, Context API, peticiones HTTP con Axios, rutas, navegación y despliegue.

El propósito de este ejercicio es comprender cómo estas herramientas pueden aplicarse en un proyecto real, especialmente en una aplicación que busca presentar servicios educativos, facilitar la navegación del usuario y permitir la interacción mediante formularios o solicitudes de información. De esta manera, se relaciona la teoría con una propuesta práctica de desarrollo web.



REST con Swagger

REST es un estilo de arquitectura utilizado para la comunicación entre aplicaciones a través de internet. Su función principal es permitir el intercambio de información entre el front-end y el back-end mediante solicitudes HTTP como GET, POST, PUT y DELETE.

En una aplicación web, REST facilita que el cliente pueda solicitar datos al servidor y recibir respuestas organizadas, normalmente en formato JSON. Gracias a esto, es posible conectar interfaces gráficas desarrolladas en ReactJS con servicios internos encargados de procesar y almacenar información.

Por otra parte, Swagger es una herramienta utilizada para documentar servicios REST. Permite visualizar de manera organizada los endpoints de una API, los tipos de solicitudes disponibles y las respuestas que puede entregar el servidor. Esto facilita el trabajo de los desarrolladores y mejora la comprensión del funcionamiento de la aplicación.

En el caso del proyecto Gamenglish, REST podría utilizarse para enviar y recibir información relacionada con formularios de contacto, servicios educativos o registros de usuarios. Por ejemplo, cuando un padre de familia diligencia un formulario para solicitar información sobre clases de inglés, el front-end envía los datos al servidor mediante una petición POST y el back-end procesa la información y genera una respuesta.



```
POST /contacto

{
  "nombre": "María López",
  "correo": "maria@gmail.com",
  "mensaje": "Deseo información sobre clases virtuales"
}
```

Figura 1. Ejemplo 1

En este ejemplo, la aplicación envía información al servidor utilizando una petición REST de tipo POST.

IBERO

UNIVERSIDAD DE CASTILLA-LA MANCHA



Planeta Formación y Universidades

VIGILADA MINEDUCACIÓN

ReactJS

ReactJS es una biblioteca de JavaScript utilizada para construir interfaces de usuario dinámicas e interactivas. Fue desarrollada por Facebook y actualmente es una de las tecnologías más utilizadas en el desarrollo front-end debido a su capacidad para crear aplicaciones web modernas, organizadas y reutilizables.

Una de las principales características de ReactJS es el uso de componentes. Estos componentes permiten dividir la interfaz en pequeñas partes independientes que pueden reutilizarse en diferentes secciones de la aplicación. Esto facilita el mantenimiento del código y mejora la organización del proyecto.

React también trabaja con un DOM virtual, lo que permite actualizar únicamente las partes de la interfaz que cambian, mejorando el rendimiento de la aplicación y haciendo que la experiencia del usuario sea más fluida.

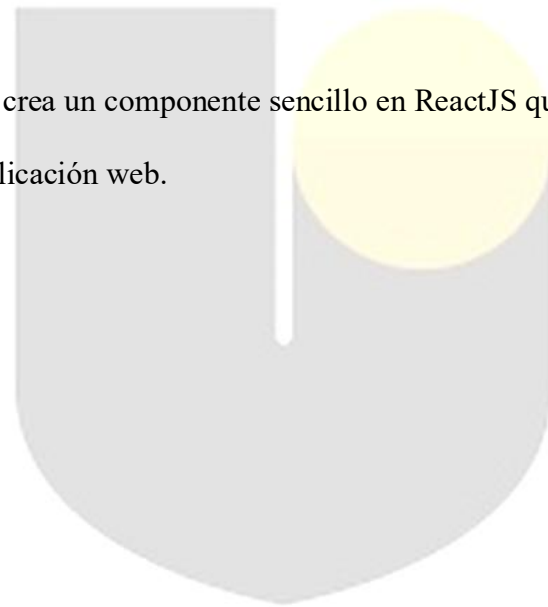
En el proyecto Gamenglish, ReactJS podría utilizarse para construir las diferentes vistas de la aplicación, como la página principal, la sección de servicios, el formulario de contacto y las páginas informativas. Gracias a los componentes, sería posible reutilizar elementos como barras de navegación, tarjetas de información o botones en distintas partes del sitio web.



```
function Bienvenida() {  
  return (  
    <h1>Bienvenido a Gamenglish</h1>  
  );  
}
```

Figura 2. Ejemplo 2

En este ejemplo se crea un componente sencillo en ReactJS que muestra un mensaje de bienvenida dentro de la aplicación web.



IBERO

UNIVERSIDAD DE LOS RÍOS



Planeta Formación y Universidades

VIGILADA MINEDUCACIÓN

Hooks

Los Hooks son funciones especiales de ReactJS que permiten utilizar características avanzadas dentro de componentes funcionales, como el manejo del estado, efectos secundarios y acceso a información compartida entre componentes. Gracias a los Hooks, el desarrollo de aplicaciones resulta más organizado y flexible.

Entre los Hooks más utilizados se encuentran useState, useContext, useEffect y useReducer, cada uno con funciones específicas dentro de la aplicación.

useState

El Hook useState se utiliza para almacenar y actualizar información dentro de un componente. Permite manejar datos dinámicos que pueden cambiar durante la ejecución de la aplicación, como textos, formularios o contadores.

En el proyecto Gamenglish, useState podría emplearse para almacenar la información ingresada por un usuario en un formulario de contacto.



```
import { useState } from "react";

function Formulario() {
  const [nombre, setNombre] = useState("");

  return (
    <input
      type="text"
      value={nombre}
      onChange={(e) => setNombre(e.target.value)}
    />
  );
}
```

Figura 3. Ejemplo 3

En este ejemplo, useState permite guardar el nombre que escribe el usuario en el campo de texto.

useContext

El Hook useContext permite compartir información entre diferentes componentes sin necesidad de enviar datos manualmente entre ellos. Esto facilita la organización de aplicaciones grandes y evita repetir código innecesario.



En Gamenglish, useContext podría utilizarse para compartir información relacionada con el usuario, el idioma seleccionado o configuraciones generales de la aplicación.

```
const UsuarioContext = React.createContext();

function Perfil() {
  const usuario = useContext(UsuarioContext);

  return <h2>{usuario.nombre}</h2>;
}
```

Figura 4. Ejemplo 4

En este caso, el componente accede a información del usuario utilizando useContext.

useEffect

El Hook useEffect se utiliza para ejecutar acciones automáticas dentro de un componente, como cargar información desde un servidor, actualizar datos o ejecutar procesos cuando cambia el estado de la aplicación.

En el proyecto Gamenglish, este Hook podría utilizarse para cargar automáticamente los servicios educativos disponibles al abrir la página principal.

```
useEffect(() => {
  console.log("La página cargó correctamente");
}, []);
```

Figura 5. Ejemplo 5



useReducer

El Hook useReducer se utiliza para manejar estados más complejos dentro de una aplicación. Su funcionamiento es similar al de useState, pero permite controlar mejor los cambios cuando existen múltiples acciones o datos relacionados.

En Gamenglish, useReducer podría utilizarse para administrar formularios más complejos o procesos relacionados con registros de usuarios y servicios.

```
const estadoInicial = { contador: 0 };

function reducer(state, action) {
  switch (action.type) {
    case "incrementar":
      return { contador: state.contador + 1 };
    default:
      return state;
  }
}
```

Figura 6. Ejemplo 6

En este ejemplo, useReducer permite controlar cambios en un contador mediante acciones específicas.



Context API

Context API es una herramienta de ReactJS que permite compartir información entre diferentes componentes de una aplicación sin necesidad de enviar propiedades manualmente en cada nivel de la estructura. Su principal objetivo es facilitar el manejo de datos globales dentro de la aplicación.

Esta herramienta resulta útil cuando varios componentes necesitan acceder a la misma información, como usuarios autenticados, configuraciones generales o temas visuales. De esta manera, se mejora la organización del código y se evita la duplicación innecesaria de datos.

En el proyecto Gamenglish, Context API podría utilizarse para compartir información relacionada con el estado del usuario, las preferencias de navegación o los datos generales de la aplicación educativa.



```
import React, { createContext } from "react";

export const TemaContext = createContext();

function App() {
  return (
    <TemaContext.Provider value={"modo claro"}>
      <Inicio />
    </TemaContext.Provider>
  );
}
```

Figura 7. Ejemplo 7

En este ejemplo, Context API permite compartir el valor “modo claro” con diferentes componentes de la aplicación sin necesidad de enviarlo manualmente en cada nivel.



Peticiones HTTP con Axios

Axios es una biblioteca de JavaScript utilizada para realizar peticiones HTTP desde el front-end hacia servidores o servicios externos. Su principal función es facilitar la comunicación entre la interfaz de usuario y el back-end de una aplicación web.

A través de Axios es posible enviar solicitudes de tipo GET, POST, PUT y DELETE, permitiendo consultar información, enviar datos, actualizar registros o eliminar contenido. Además, esta herramienta facilita el manejo de respuestas y errores, haciendo más organizada la interacción con APIs y servicios REST.

En el proyecto Gamenglish, Axios podría utilizarse para enviar la información ingresada por los usuarios en el formulario de contacto hacia el servidor, o para consultar dinámicamente los servicios educativos disponibles dentro de la aplicación.



```
import axios from "axios";

axios.get("http://localhost:3000/servicios")
  .then((respuesta) => {
    console.log(respuesta.data);
  })
  .catch((error) => {
    console.log(error);
  });
```

Figura 8. Ejemplo 8

En este ejemplo, Axios realiza una petición GET para consultar información de servicios disponibles en la aplicación web.



Rutas y navegación

Las rutas y la navegación permiten organizar una aplicación web en diferentes páginas o vistas, facilitando el desplazamiento del usuario dentro del sistema sin necesidad de recargar completamente el sitio web.

En ReactJS, este proceso suele realizarse mediante bibliotecas como React Router, la cual permite definir rutas para acceder a diferentes componentes de la aplicación. Gracias a esto, es posible construir aplicaciones más dinámicas y con una mejor experiencia de usuario.

Las rutas ayudan a estructurar el contenido de manera organizada, permitiendo que cada sección tenga su propia dirección y funcionalidad. Por ejemplo, una aplicación puede tener rutas independientes para inicio, servicios, contacto o perfil de usuario.

En el proyecto Gamenglish, las rutas podrían utilizarse para permitir la navegación entre la página principal, la sección de servicios educativos, el formulario de contacto y otras áreas informativas relacionadas con el aprendizaje del inglés.



```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Inicio />} />
        <Route path="/servicios" element={<Servicios />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Figura 9. Ejemplo 9

En este ejemplo, se definen rutas para navegar entre la página de inicio y la sección de servicios dentro de la aplicación web.



Planeta Formación y Universidades

VIGILADA MINEDUCACIÓN

Despliegue

El despliegue es el proceso mediante el cual una aplicación web es publicada en un servidor o plataforma en línea para que pueda ser utilizada por los usuarios a través de internet. Esta etapa representa el paso final del desarrollo, ya que permite que la aplicación deje de ejecutarse únicamente en un entorno local y esté disponible públicamente.

El despliegue es importante porque facilita el acceso a la aplicación desde diferentes dispositivos y ubicaciones, permitiendo además realizar pruebas reales de funcionamiento y experiencia de usuario.

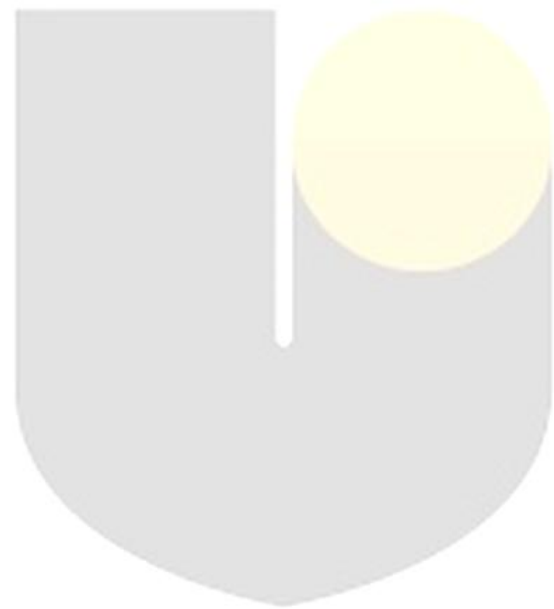
Actualmente existen diferentes plataformas que permiten desplegar aplicaciones web de manera sencilla, entre ellas GitHub Pages, Vercel, Netlify y servicios en la nube como Google Cloud o AWS.

En el caso del proyecto Gamenglish, el despliegue permitiría que padres de familia y usuarios interesados puedan acceder a la aplicación web desde internet para consultar los servicios educativos, navegar entre las diferentes secciones y comunicarse mediante formularios de contacto.



<https://gamenglish.vercel.app>

Figura 10. Ejemplo 10



IBERO

UNIVERSIDAD CATÓLICA DE CHILE



Planeta Formación y Universidades

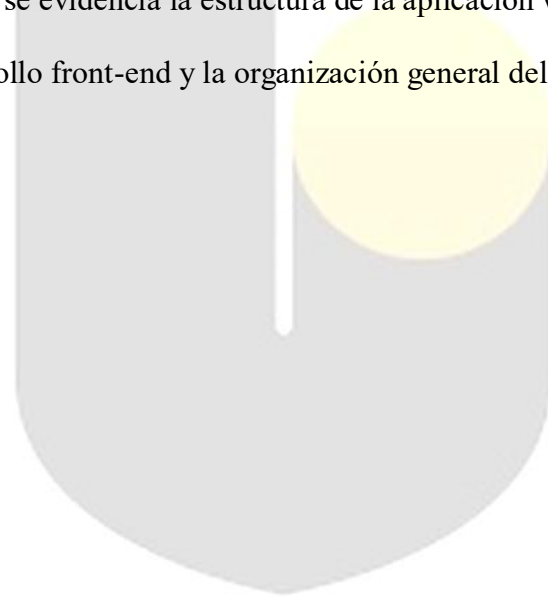
VIGILADA MINEDUCACIÓN

URL del Código de la Aplicación Web

El código fuente de la aplicación web Gamenglish se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/pilytrianar/gamenglish>

En este repositorio se evidencia la estructura de la aplicación web, incluyendo elementos relacionados con el desarrollo front-end y la organización general del proyecto.



IBERO

UNIVERSIDAD DE LOS RÍOS



Planeta Formación y Universidades

VIGILADA MINEDUCACIÓN

Conclusión

El desarrollo de esta actividad permitió comprender la importancia de las tecnologías front-end y back-end en la construcción de aplicaciones web modernas. A través del proyecto Gamenglish fue posible analizar cómo herramientas como ReactJS, Hooks, Context API, Axios y Swagger facilitan la creación de aplicaciones dinámicas, organizadas y funcionales.

Asimismo, se logró evidenciar la manera en que el front-end y el back-end trabajan de forma integrada mediante servicios REST, permitiendo la comunicación entre la interfaz de usuario y el servidor. Del mismo modo, el uso de rutas, navegación y manejo de estado contribuye a mejorar la experiencia del usuario y la estructura general de la aplicación.

Finalmente, esta actividad permitió fortalecer conocimientos relacionados con el desarrollo web moderno, comprendiendo no solo la parte teórica de las tecnologías utilizadas, sino también su aplicación práctica dentro de un proyecto real orientado al ámbito educativo.



Bibliografía

Pérez Ibarra, S. G., Quispe, J. R., Mullicundo, F. F., & Lamas, D. A. (2021). *Herramientas y tecnologías para el desarrollo web desde el frontend y backend*. XXIII Workshop de Investigadores en Ciencias de la Computación.

React. (2026). *React Documentation*. <https://react.dev>

Axios. (2026). *Axios Documentation*. <https://axios-http.com>

Swagger. (2026). *Swagger Documentation*. <https://swagger.io>

IBERO

UNIVERSIDAD DE LA GUAYANA



Planeta Formación y Universidades

VIGILADA MINEDUCACIÓN